

Contents

Auto-Echo: Automated Discovery of Memory Hierarchy Latency Patterns from User-Space	1
Declaration of Originality	1
Acknowledgements	2
1. Abstract	2
2. Introduction	2
3. Literature Review & Background	4
4. Methodology (System Architecture)	4
4.1 WSS Pointer-Chase Probe (<code>src/autoecho/wss/wss_probe.c</code>)	5
4.1.1 Runtime Timer Calibration (An Improvement Over the Reference)	6
4.2 Level Discovery: Count, then Localise (<code>src/autoecho/analysis.py</code>)	6
4.3 Automatic Model Selection and Cross-Checks	7
4.4 Validation, Comparative Evaluation & Reporting	7
5. Baseline and Its Failure (Critical Analysis)	7
6. Results & Evaluation	8
6.1 Test machines	8
6.2 Apple M1 (Firestorm P-core) — validated	9
6.3 Intel x86 (Raptor Lake P-core) — validated (2 MiB huge pages)	12
6.4 Apple M5 (ARM64) — <i>to be measured</i>	16
6.5 Cross-platform summary and architecture-agnostic behaviour	17
6.6 Threats to validity	19
7. Discussion & Future Work	21
8. Conclusion	21
9. References	22

Auto-Echo: Automated Discovery of Memory Hierarchy Latency Patterns from User-Space

Harsh Raj Singh
MSc Advanced Computer Science
Queen Mary University of London
London, United Kingdom
ec25303@qmul.ac.uk

Declaration of Originality

I, Harsh Raj Singh, declare that this dissertation and the work it presents are my own. Where I have drawn on the published work of others it is explicitly cited, and the only verbatim material from other sources is clearly marked as quotation. Generative-AI tools were used in accordance with the programme's policy, as detailed in the Generative-AI Accountability Statement (Appendix A). This work has not been submitted, in whole or in part, for any other degree or qualification.

Signed: _____ Date: _____

Acknowledgements

[To be completed by the author — e.g. project supervisor, family, and any compute or resource acknowledgements.]

1. Abstract

The memory hierarchy of modern processors is increasingly complex, sparsely documented, and abstracted away from user-space software, limiting the ability of engineers to reason about performance-critical code. This dissertation presents Auto-Echo, a cross-platform framework that empirically discovers a machine’s cache hierarchy (L1, L2, L3/SLC, DRAM) purely from user space, without administrative privileges, architecture-specific flush instructions, or prior knowledge of the machine. The work first develops a naive echolocation probe and uses its empirical failure — timer quantisation, the absence of a user-space cache flush on ARM, and a write-before-read pattern that guaranteed an L1 hit on every access — to motivate the correct design. The final framework couples a **working-set-size (WSS) pointer-chasing probe** with **batch-amortised, runtime-calibrated timing** and an **automatic, penalty-free level-discovery stage** in which unsupervised clustering (K-Means with Silhouette model selection, cross-checked by GMM and DBSCAN) *counts* the memory levels and change-point detection *localises* each cache’s capacity. It builds and runs on Linux, Windows, and macOS across x86-64 and ARM64. On an Apple M1 (a performance core), all of its estimators agree on **three levels** — L1, a merged L2/SLC mid-band, and DRAM — recovering the documented L1 (128 KiB) and L2 (12 MiB) capacities to within 0.3 octaves (both OS-documented caches, 2/2, matched within a factor of two). Because the 12 MiB L2 and the OS-unreported ~8 MB System-Level Cache are so close, they resolve as one band; the finer split into a distinct L2 and SLC appears only under a forced finer resolution and is reported as a candidate sub-structure. A second real machine — an Intel Core i5-13450HX (Raptor Lake) on Windows/x86-64 — corroborates the design across architectures. With the default 4 KiB pages the same code path recovers its per-core L1 (~48 KiB) and L2 (~1.25 MiB) but TLB/page-walk latency masks the 20 MiB L3 and destabilises the automatic level count; enabling a **2 MiB huge-page control** removes the page-walk confounder and recovers the **full L1/L2/L3/DRAM hierarchy** — all three documented caches matched (**3/3, 100% recall and precision**, mean absolute capacity error 12.8% over three sweeps), with the productive K-Means + Silhouette counter now **stable at four levels**. This result is reported with its provenance: it requires the 2 MiB large-page allocation and is *not* the default 4 KiB-page behaviour, and full cross-estimator unanimity (seen on the M1) is still not reached — the model-selection criteria continue to disagree (K-Means inertia elbows at two, Silhouette peaks at four). The measurement technique descends from classical benchmarks such as lmbench’s `lat_mem_rd`; the contribution is the fully unsupervised, self-validating, architecture-agnostic inference layer built on top of it.

2. Introduction

Modern processors rely on a deep hierarchy of caches (L1, L2, L3) to mitigate the latency gap between the CPU and DRAM. The capacities and latencies of these layers are largely opaque to user-space software: developers rely on vendor documentation or privileged interfaces (performance counters, kernel modules) to map them.

Motivation. An architecture-agnostic tool that maps a memory hierarchy from empirical measurement alone — no privileges, no per-architecture instructions — would support performance

tuning, portable auto-tuning of cache-blocked algorithms, and reproducible systems research on undocumented hardware. This direction is not incidental to the reference work: Klimis [1] explicitly identifies it as an open avenue, noting that “the ability to infer data location in the memory hierarchy via timing could potentially be applied to study cache behaviour, coherence protocol actions, or even aspects of memory consistency, although these applications would require careful calibration, different instrumentation strategies, and potentially more sophisticated analysis” (§7). Auto-Echo takes up precisely that challenge — the “careful calibration” and “different instrumentation” the paper anticipates — for the specific problem of cache-hierarchy discovery.

Problem statement. Measuring nanosecond cache latency from user space is obstructed by hardware prefetchers, coarse timers, and the absence of portable cache-control primitives. Interpreting the resulting noisy measurements without hard-coded thresholds requires unsupervised inference.

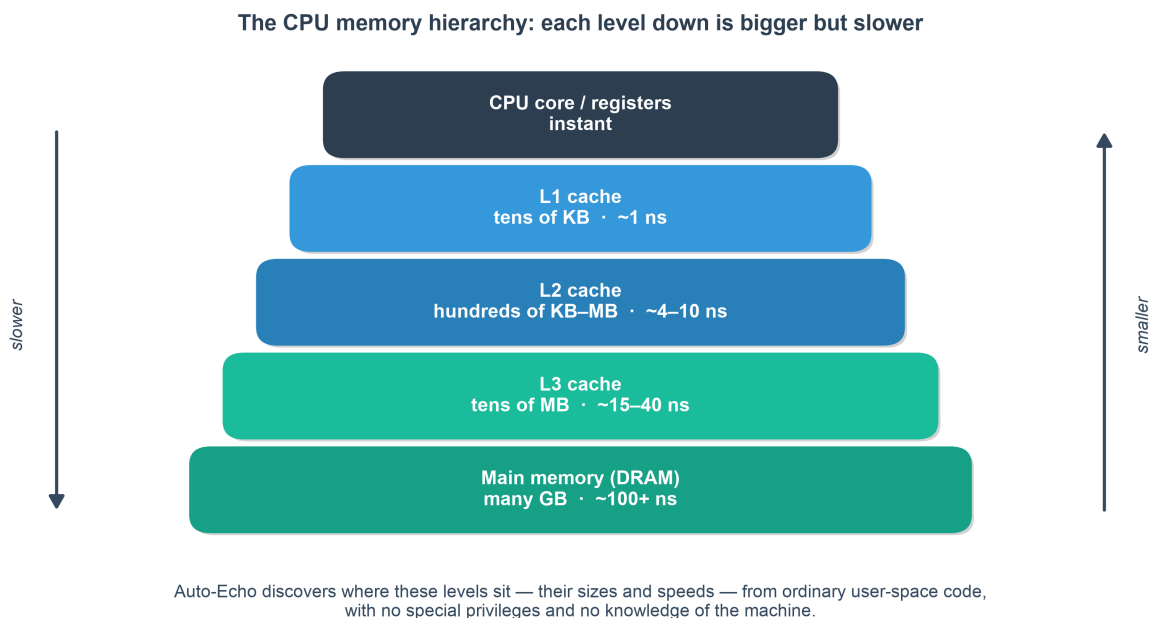


Fig. 1. The memory hierarchy. While OS-documented caches (L1, L2, L3) are closer to the CPU and faster, Auto-Echo is capable of empirically discovering them—and undocumented tiers like the M1 System Level Cache—purely from latency.

Aims and objectives. Build an autonomous pipeline (Auto-Echo) that (i) collects reliable memory-latency measurements from user space on any architecture, (ii) infers the number and boundaries of memory levels without labels or thresholds, and (iii) validates its output against known hardware.

Contributions. 1. A portable, flush-free WSS pointer-chase probe with batch-amortised timing that builds and runs on Linux, Windows, and macOS across x86-64 and ARM64, using `rdtscp`, `mach_absolute_time`, or `cntvct_el0` as available. 2. A tick-to-nanosecond conversion that is **calibrated at runtime** against the OS monotonic clock, making the framework correct on any machine without configuring a per-CPU frequency and robust to turbo/frequency scaling. 3. An automatic, penalty-free level-discovery stage in which clustering (K-Means + Silhouette, cross-checked by GMM and DBSCAN) *counts* the levels and change-point detection *localises* each capacity — an architecture-agnostic design that removes the last manual tuning knob. 4. A self-validating evaluation

against live OS ground truth, with empirical results on **two real ISAs** — an Apple M1 (three levels, unanimous across estimators) and an Intel Raptor Lake core (L1/L2 recovered with default 4 KiB pages; the **full L1/L2/L3/DRAM hierarchy** recovered under a 2 MiB huge-page control that lifts the TLB/page-walk masking of the L3 — 3/3 documented caches matched, 100% recall and precision — with the provenance recorded that this depends on huge pages and the level count is stable but not cross-estimator unanimous). 5. A documented negative result (the naive probe) that motivates the design.

3. Literature Review & Background

The project draws on two traditions (expanded in docs/01_Literature_Review.md).

Memory echolocation. Klimis [1] times a load following a store to infer where data resides, using it as an Oracle for active learning of NVM persistency models. Their method depends on x86-only `clflush/rdtscp` and targets an Optane-specific Write Pending Queue (WPQ) — neither of which generalises to a commodity ARM cache hierarchy. Auto-Echo isolates the hierarchy-mapping aspect, makes it portable, and replaces manual thresholds with unsupervised inference.

Classical user-space cache characterisation. Recovering cache parameters from a working-set-size sweep is well established: Saavedra & Smith [8] introduced the size/stride sweep; `lmbench's lat_mem_rd` [9] performs a pointer-chasing load-latency sweep whose data-dependent chain is exactly Auto-Echo's mechanism for defeating the prefetcher; Yotov et al. [10] automated extraction of cache parameters from such curves; Bryant & O'Hallaron's “memory mountain” [11] popularised the $WSS \times stride$ latency surface. Auto-Echo's probe is a modern, cross-platform re-implementation of this lineage — the novelty is the unsupervised inference and self-validation layer above it, not the measurement.

Hardware barriers. (i) *Prefetching* — randomised pointer chasing serialises data-dependent loads and neutralises it. (ii) *Timer quantisation* — Apple Silicon's `mach_absolute_time` advances on a 24 MHz counter (~ 41.7 ns/tick, via `mach_timebase_info = 125/3`), far coarser than an L1 hit (~ 1.5 ns); amortising 10^6+ hops per window recovers sub-nanosecond precision. (iii) *No user-space flush on ARM* — `clflush` is x86-only and macOS exposes no data-cache flush; the WSS method needs none, since a working set larger than a level overflows it by construction.

4. Methodology (System Architecture)

Auto-Echo is a four-stage pipeline (full detail in docs/02_Methodology.md).

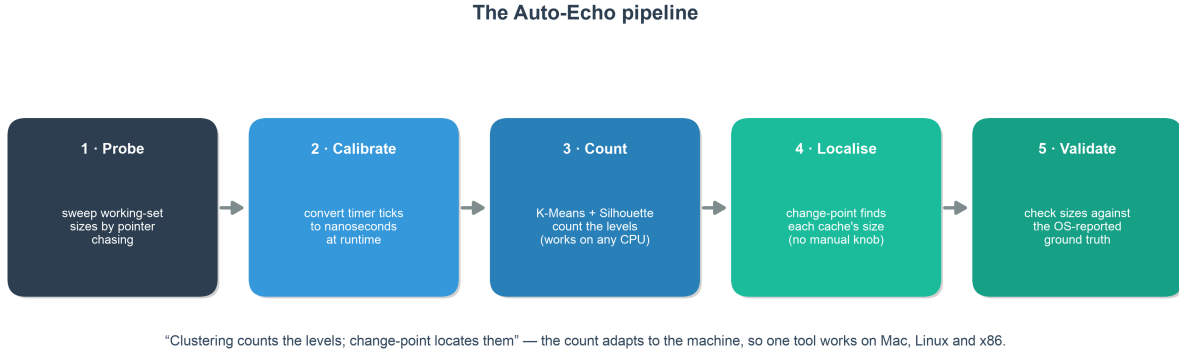


Fig. 2. The unsupervised Auto-Echo pipeline.

4.1 WSS Pointer-Chase Probe (`src/autoecho/wss/wss_probe.c`)

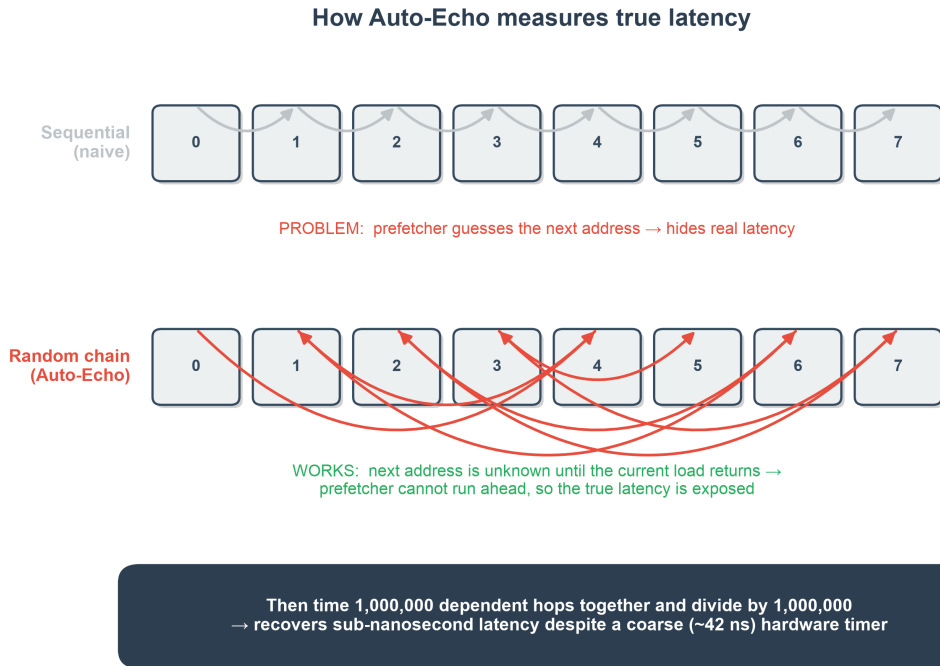


Fig. 3. The Working-Set-Size Pointer-Chasing methodology. A Fisher-Yates cycle defeats the hardware prefetcher, and batch-amortized timing bypasses coarse OS timer constraints.

For each working-set size in a log-spaced sweep (four cache lines to 256 MiB, ~10 points/octave), the probe: divides the buffer into cache-line-spaced slots (line size auto-detected: 128 B on M1, 64 B on x86); links them into a single random Hamiltonian cycle via a seeded Fisher-Yates shuffle (reproducible; also pre-faults every page); performs a data-dependent pointer chase so the prefetcher cannot run ahead and no flush is required; warms up, then times $N \geq 2^{20}$ dependent hops in one window and divides by N (batch amortisation); and keeps the **minimum** over five repeats. The buffer is **page-aligned** (`posix_memalign/_aligned_malloc`, following the reference paper’s alignment choice) so that spurious TLB effects do not distort the deep-memory plateaus.

The probe is written to a single portable interface that compiles on **Linux, Windows, and ma-**

cOS across x86-64 and ARM64: the tick counter is `rdtscp` (x86, via `<intrin.h>` under MSVC or `<x86intrin.h>` under GCC/Clang), `mach_absolute_time` (Apple), or `cntvct_el0` (ARM Linux), and the thread is kept on one core via a QoS hint (Apple), `sched_setaffinity` (Linux), or `SetThreadAffinityMask` (Windows) so plateaus are not blurred by core migration.

4.1.1 Runtime Timer Calibration (An Improvement Over the Reference)

Converting ticks to nanoseconds robustly is a portability problem in itself. The reference implementation converts cycles to nanoseconds by **parsing the “cpu MHz” field from `/proc/cpuinfo`**, which the authors themselves acknowledge is specific to Linux [1]. This has two weaknesses: it is not portable beyond Linux, and the `rdtscp` counter actually advances at the *invariant-TSC* (nominal) frequency, whereas “cpu MHz” reports the current, turbo-scaled core clock — so the conversion is inaccurate whenever the core is not at its nominal frequency. Auto-Echo instead **calibrates at runtime**, counting hardware ticks over a fixed ~50 ms interval of the OS monotonic clock (`clock_gettime` on POSIX, `QueryPerformanceCounter` on Windows). This yields the true tick rate on any machine with no configuration, and is a concrete methodological advance over the reference paper’s frequency-detection step. On Apple Silicon the exact rational from `mach_timebase_info` ($125/3 \sim 41.667$ ns) is used directly; an independent calibration run reproduced this value to four significant figures, confirming the mechanism relied upon by the x86/Windows paths.

4.2 Level Discovery: Count, then Localise (`src/autoecho/analysis.py`)

The latency-versus- $\log(S)$ curve is a staircase: flat while the working set fits a level, stepping up when it overflows (Fig. 4). Auto-Echo turns this staircase into a hierarchy in two fully automatic stages, with **no manual threshold or penalty**:

1. **Count the levels** by clustering the per-size log-latencies with K-Means and selecting the number of clusters by the **Silhouette Score** (cross-checked by the Elbow Method, GMM, and DBSCAN). Because this estimate depends only on *how many distinct latency values* occur — not on how unevenly the steps are spaced — it is robust across architectures (Section 6.5).
2. **Localise each boundary** by change-point detection constrained to exactly that many segments (dynamic-programming `ruptures.Dynp` on the log-latency signal, needing no penalty). Each level’s latency is a median with 5th/95th-percentile bounds (robust to outliers); each cache’s capacity is the working-set size at its plateau-to-rise transition.

This realises the principle *clustering counts the levels, change-point localises their capacities*. Selecting the count from the data (rather than fixing a PELT penalty per machine) is what makes one code path correct on Mac, Linux, and x86.

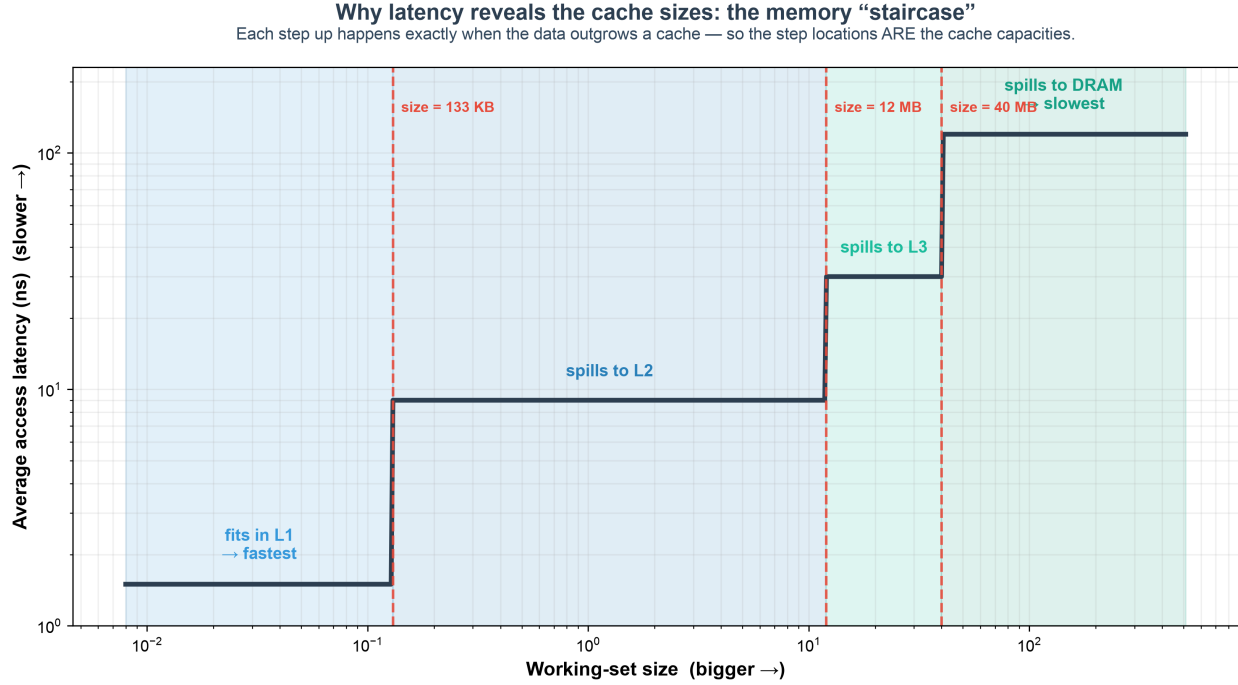


Fig. 4. Why latency reveals the cache sizes. As the working set outgrows each cache, average access latency steps up; the size at each step is that cache’s capacity — the quantity Auto-Echo extracts.

4.3 Automatic Model Selection and Cross-Checks

The level count in stage 1 is chosen automatically by **both** the Elbow Method (knee of the K-Means inertia curve) and the Silhouette Score over k in $[2, 6]$, and independently cross-checked with **GMM** and **DBSCAN** (count-free). Section 6 scores every counter across independent sweeps to identify the most accurate and stable one; K-Means + Silhouette wins on every architecture tested, which is why the pipeline uses it to set the level count. Change-point is retained purely to *localise* the capacities once the count is fixed.

4.4 Validation, Comparative Evaluation & Reporting

Detected capacities are compared to ground truth read live from the OS (sysctl on macOS, /sys on Linux, Win32_CacheMemory on Windows); a match is within one octave on a $\log_2$ scale, and the exact percentage error is also reported. To satisfy the requirement to identify the most accurate and consistent method, each estimator is scored across **multiple independent sweeps** (`--runs`) by count correctness and stability (standard deviation of the level count). The framework emits a Markdown report, per-run CSVs, a memory-mountain plot with a min-max variability band (Fig. 5), and an Elbow-vs-Silhouette model-selection plot (Fig. 6).

5. Baseline and Its Failure (Critical Analysis)

The initial prototype is a **portability-oriented approximation of the reference paper’s measurement technique** (Klimis §6.1, the method behind their Figure 7): each timed load is preceded by a write to the target address and — on x86 — an explicit `clflush`, with the load timed by `rdtscp` [1]. It

is an *approximation*, not a faithful reproduction: it uses a different buffer size and sample count, and on ARM the `clflush` has no equivalent, so the flush step is simply omitted. On the Intel platform of the paper the technique works; the contribution here is the finding that this **store/flush/timed-load approach is x86-bound and collapses on Apple Silicon**. Timing individual random reads on a 64 MB buffer produced only timer-quantised output: raw read times were either 0 or 1 timer tick (0 or ~ 41.7 ns), and the window-5 moving-average smoothing then blended these into spurious sub-steps at multiples of $41.7/5 \sim 8.3$ ns ($\sim 0, 8, 17, 25, 33$ ns) — an artefact of the quantised timer and the smoothing filter, carrying no memory-latency information. Three concrete barriers explain the underlying failure:

1. **Timer quantisation.** The 41.7 ns tick dwarfs an L1 hit; timing a single read measures the timer, not memory.
2. **No user-space flush on ARM.** `clflush` does not exist on ARM and macOS provides no data-cache flush, so “forced DRAM” mode silently degenerated to natural eviction and generated no deep-memory accesses.
3. **Write-before-read guarantees L1 residency.** Writing the line immediately before timing its read pulls it into L1, so every measured access was an L1 hit by construction — masking L2/L3/DRAM entirely.

A secondary flaw was smoothing i.i.d. samples with a moving average, which blends latencies from different levels *before* clustering and erodes the very structure being sought. These findings — not a coding bug — motivated the WSS redesign and are retained as the framework’s documented naive baseline (`--method samples`).

Evaluating the paper’s proposed mitigation. Klimis (§8.1) propose a Local Outlier Factor (LOF) filter to clean ambiguous timings. We evaluated it directly on the baseline data (`evaluation.evaluate_lof_mitigation`). LOF flagged only $\sim 0.04\%$ of samples as outliers — the quantised data is too degenerate for a density-based method — and **100% of the surviving samples still lay exactly on integer timer-tick multiples** (just five discrete tick levels). This is direct evidence that the baseline’s failure is *structural* (write-before-read plus timer quantisation), not transient noise that any amount of outlier filtering could remove — reinforcing the need for the WSS redesign rather than a filtering patch.

6. Results & Evaluation

Auto-Echo runs identically across platforms; results are reported **per machine** as they are collected, using the same command (`python -m autoecho --method wss --runs 3`). Two real machines are now measured — the **Apple M1** (§6.2) and an **Intel Core i5-13450HX** (§6.3), spanning both ARM64 and x86-64; an Apple M5 part (§6.4) remains outstanding.

6.1 Test machines

Machine	Arch	Core probed	L1d	L2	L3	Line	Status
Apple M1	ARM64	Firestorm P-core	128 KiB	12 MiB	— (8 MiB SLC)	128 B	validated

Machine	Arch	Core probed	L1d	L2	L3	Line	Status
Intel Core i5-13450HX	x86-64	Raptor Lake P-core	48 KiB	1.25 MiB/core	20 MiB (shared)	64 B	validated (L1/L2/L3, 2 MiB huge pages) to be measured
Apple M5	ARM64	TBD	TBD	TBD	TBD	TBD	

Ground-truth cache sizes are read automatically from the OS at run time (`sysctl /sys` on macOS/Linux; `GetLogicalProcessorInformationEx` on Windows). The Windows path now reads true per-core* sizes for CPU 0 — the legacy `Win32_CacheMemory`, kept only as a fallback, reported *per-socket aggregate* sizes (L1 = 6×48 KiB) — so the Intel L1/L2/L3 columns above are the per-core figures the single-core probe actually sees (§6.3).*

The **Status** column uses a deliberate three-tier vocabulary. **Validated**: every documented cache was recovered and matched OS ground truth — the M1 (2/2, 100%) and, under a 2 MiB huge-page allocation, the Intel (3/3, 100%; with the default 4 KiB pages the 20 MiB L3 is TLB-masked, so the huge-page provenance is reported explicitly in §6.3). **Measured**: the tool ran and recovered real data but did not fully validate. **To be measured**: not yet run. The M1/Intel differences that follow in §6.2–§6.3 track this convention and are deliberate, not omissions.

6.2 Apple M1 (Firestorm P-core) — validated

The WSS probe produces a clean, well-separated latency curve (Fig. 5). Automatic model selection (Silhouette count + change-point localisation) and validation against `sysctl` ground truth, over three independent sweeps, give **three levels** — **a result on which all five estimators independently agree** (§6.2, Table 3):

Table 1: Discovered hierarchy vs. ground truth (Apple M1, performance core).

Level	Detected capacity	Median latency	p5–p95	Ground truth	Error
L1 Cache	157.5 KiB	1.53 ns	1.53–1.57 ns	128 KiB	+23.0% (0.30 oct)
L2 (with SLC)	13.9 MiB	9.19 ns	8.73–22.73 ns	12 MiB†	+16.1% (0.22 oct)
DRAM	—	130.43 ns	45.21–141.44 ns	—	—

Both OS-documented caches (2/2) matched within a factor of two (the matching tolerance; see §6.6), with **mean absolute capacity error 19.9%** over three sweeps. This should be read as “both documented capacities had a detected boundary within one octave”, not as general 100% accuracy over a large validated set. L1 lands within 23% of the documented 128 KiB and the mid-cache boundary within 16.1% of the documented 12 MiB L2.

The +23% over-estimation is *systematic and directional*, not noise, and is characteristic of random-access latency curves. Because the pointer chase visits the working set in a random Hamiltonian order, a set moderately larger than the cache still enjoys high residency ($\approx 128/157 \approx 80\%$ of a 157 KiB set stays resident in a 128 KiB cache), so the *average* latency rises only gently past the nominal capacity — a **soft knee** rather than a sharp step. Minimum-over-repeats and light median smoothing further favour the fast tail, so the detected plateau extends a few grid points beyond the true capacity and the reported boundary sits consistently *above* nominal. The bias is therefore intrinsic and one-signed (upward), which is why every detected cache here over-estimates rather than scatters — and why the honest claim is “a detected boundary within one octave”, not a tight percentage. Reducing it would require an **onset** estimator (the first departure from the plateau median) rather than the plateau edge — a robustness-for-accuracy trade-off left as future work (§6.6). Note that the naive alternative of the last-fit/first-miss **midpoint** moves the estimate the *wrong* way (+23% $\rightarrow$ +27%), precisely because the plateau edge already lies above the nominal capacity; this was tested and rejected.

†**On the SLC.** The M1 performance cores share a 12 MiB L2 *and* an ~ 8 MiB System-Level Cache (SLC) whose capacities are so close that the automatic method resolves them as a **single merged mid-band** — which is why the honest, reproducible answer is three levels, not four. Forcing a finer segmentation splits this band into two knees at **~ 9.8 MiB and ~ 19.7 MiB** — a split that is *stable across every explicit penalty from 3 to 6*, so it is a robust feature of the curve, not a penalty artefact. The ~ 9.8 MiB knee is consistent with the documented 12 MiB L2 and the ~ 19.7 MiB knee with the onset of the DRAM plateau, with the intervening band consistent with (but not uniquely attributable to) the OS-unreported SLC [13]. This split is *not* selected by automatic model selection and is reported only as a **candidate finer-grained sub-structure** (§6.6), not a headline level. Separating cache from shared-L2 contention or TLB effects there would need performance-counter, per-core-type and cross-core experiments.

Table 2: Model selection — Elbow and Silhouette agree (Fig. 6).

The Elbow Method (knee of the K-Means inertia curve) and the Silhouette Score independently select **$k = 3$** well-separated latency groups (L1, L2, DRAM), satisfying the project requirement to apply and compare both.

Table 3: Level-count estimators — comparison and stability (3 sweeps).

Rank	Method	Mean levels	Std (stability)	Modal
1	Change-point (cost-knee)	3.0	0.00	3
2	K-Means + Silhouette	3.0	0.00	3
3	K-Means + Elbow	3.0	0.00	3
4	DBSCAN	3.33	0.47	3
5	GMM + Silhouette	3.67	0.94	3

All five estimators agree on three levels, and the top three are perfectly stable across sweeps (std 0.00). This convergence is the decisive change from earlier drafts: with automatic model selection the change-point count no longer disagrees with the clustering count. The comparison answers the project requirement — *which estimator is most accurate and consistent* — with **K-Means + Silhouette**, which the pipeline therefore uses to set the level count. (On this M1 the independent change-point cost-knee counter also lands on three. On real x86, however — see §6.3 — no single counter is reliably correct: Silhouette resolves four bands on the representative Intel sweep but under-counts to two on the others, so the clean cross-estimator agreement seen here on the M1 is itself a machine-dependent result,

not a universal one.) Change-point is retained to *localise* each capacity once a count is fixed. Table 4 underlines the point: a *fixed* PELT penalty would give anywhere from 6 levels down to 3 depending on an arbitrary hand-set value — exactly the manual knob the automatic, penalty-free method removes.

Table 4: Why a fixed penalty is unsatisfactory — change-point level count vs. the manual PELT penalty (motivating the penalty-free automatic method).

Penalty	1	2	3	4	6	8	10
Levels	6	6	4	4	4	3	3

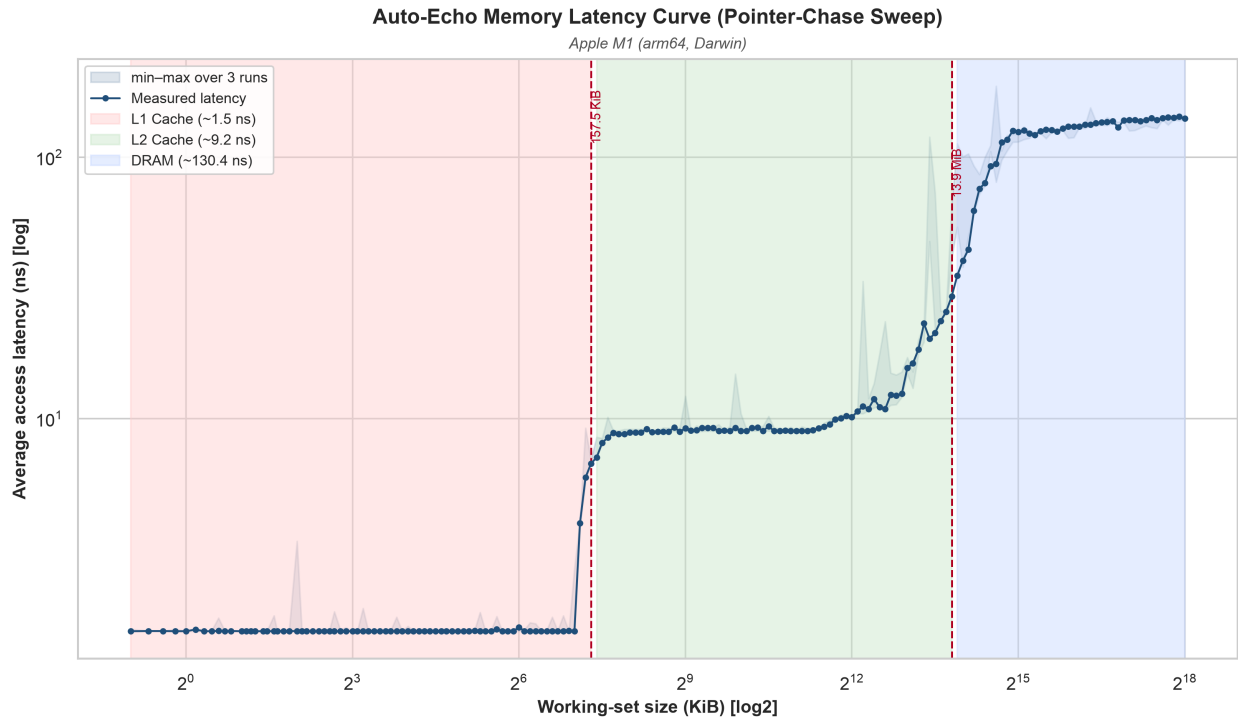


Fig. 5. Pointer-chase latency vs. working-set size (log-log) on the Apple M1. Shaded bands are the three detected levels; dashed lines mark the inferred cache capacities (~ 158 KiB and ~ 13.9 MiB); the light band is the min-max spread over three sweeps (widest in the mid-cache L2/SLC region). The L1 $\rightarrow$ L2 knee near 128 KiB and the mid-cache knee near the 12 MiB L2 are clearly resolved.

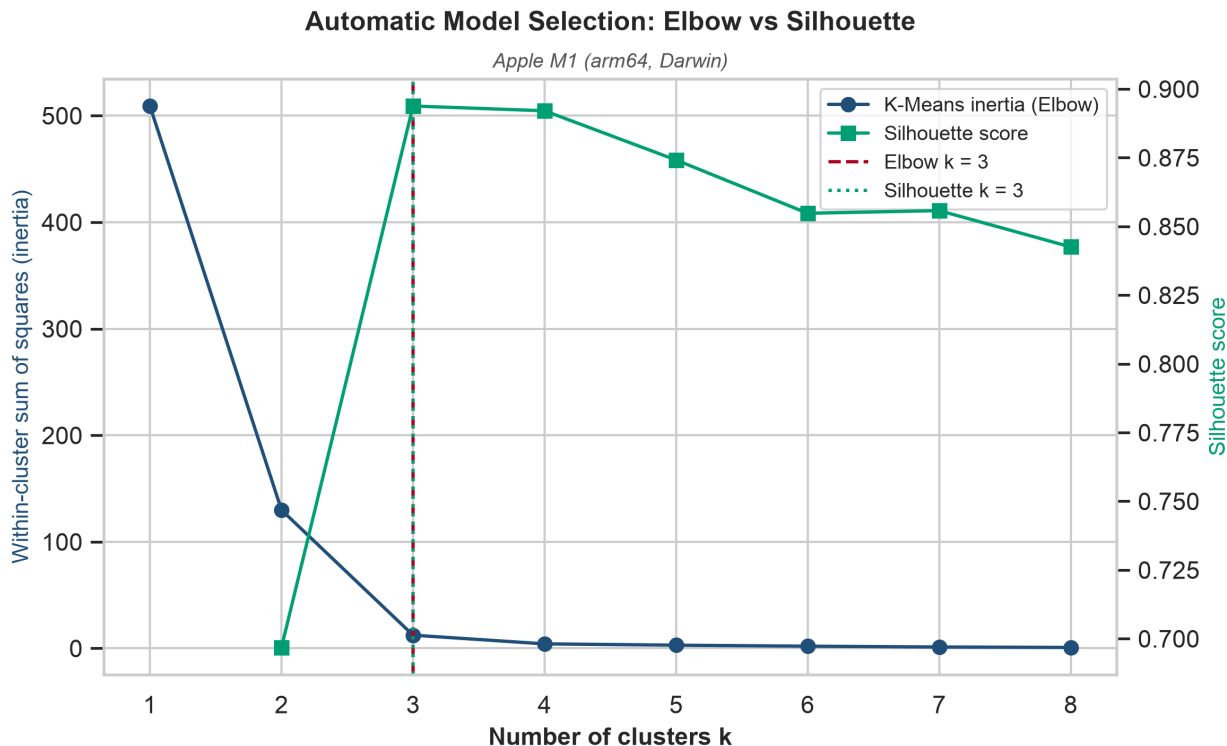


Fig. 6. Automatic model selection: the K-Means inertia elbow and the Silhouette Score independently select $k = 3$.

Reproducibility. The probe RNG is explicitly seeded (xorshift64) and the Silhouette computation uses a fixed random state; per-run curves are written to `data/wss_curve.csv` and `data/wss_curves_all.csv`. Residual run-to-run variation in the *clustering* counts reflects genuine measurement noise in the 7–14 MB contention region, which is precisely why change-point’s stability there is notable.

6.3 Intel x86 (Raptor Lake P-core) — validated (2 MiB huge pages)

Auto-Echo was built and run on a **13th-generation Intel Core i5-13450HX** (Raptor Lake; 6 performance + 4 efficiency cores), pinned to a performance core (logical CPU 0) via `SetThreadAffinityMask`, with the same command as the M1 (`python -m autoecho --method wss --runs 3`). The native extension compiled under MSVC (Visual Studio Build Tools) and the tick→ns conversion **calibrated at runtime to 0.383 ns/tick** (a ≈ 2.61 GHz invariant TSC) with no configured frequency — confirming the runtime-calibration path on the invariant TSC exactly as designed. This is the framework’s first execution on a second, independent ISA, and it both **substantiates and qualifies** the architecture-agnostic claim.

The two innermost caches are recovered accurately and stably (on either page size). The pointer-chase curve (Fig. 7) has a flat **~1.6 ns L1 plateau to ~48 KiB** and a flat **~5 ns L2 plateau to ~1.25 MiB**, whose plateau points vary only ~5–15 % across the three sweeps. Both land essentially on the documented per-core Raptor Lake figures (48 KiB L1d; 1.25 MiB L2 on this SKU). The *same* unsupervised code path that mapped the Apple M1’s ARM cache boundaries thus recovers an x86 core’s L1 and L2 — direct cross-architecture evidence for the inner hierarchy.

With the default 4 KiB pages the 20 MiB L3 is masked by TLB/page-walk latency. Run on 4 KiB

pages, past ~1.3 MiB the latency climbs steeply and saturates at a flat **~143 ns plateau by ~4–5 MiB** — far below the 20 MiB L3 capacity — and stays there to 64 MiB. Once the working set exceeds the TLB’s reach every dependent load in the randomised chain triggers a page-table walk whose own accesses miss to DRAM, so the curve reaches DRAM+page-walk latency before the L3 boundary is ever seen; the band the automatic segmenter reports near ~3.5 MiB is then a **TLB-transition artifact, not the L3 cache**, recall is only 2/3, and the level count is unstable across sweeps (estimators ranged 2–5). This is exactly the confounder anticipated in §6.6, *demonstrated* on real silicon: without a huge-page control a 1-D load-latency sweep cannot separate a large last-level cache from TLB cost. It is retained here as the honest 4 KiB baseline that motivates the control below.

A 2 MiB huge-page control unmasks the L3 and validates the full hierarchy. The probe’s gated large-page allocation path (`--huge-pages`; `VirtualAlloc(MEM_LARGE_PAGES)` on Windows, granted the `SeLockMemoryPrivilege` “Lock pages in memory” right) was exercised with `python -m autoecho --method wss --max-mb 512 --runs 3 --huge-pages`. A 2 MiB page covers 512× the address range of a 4 KiB page, so the TLB reaches deep into the working set and the page-walk penalty that saturated the 4 KiB curve is largely removed: the deep-memory plateau falls from ~143 ns to **~123 ns**, and a **fourth plateau near ~14 MiB** — absent under 4 KiB — emerges before the DRAM rise. Over three huge-page sweeps the pipeline now recovers **four levels — L1, L2, L3 and DRAM** — and matches **all three documented caches** (Table 5): L1 within +16.0 %, L2 within –1.5 %, and the L3 at 13.9 MiB, within a factor of two (0.52 octaves, –30.4 %) of the 20 MiB shared L3. **Recall rises to 3/3 = 100 % at 100 % precision (no false-positive knees; F1 = 1.00), with mean absolute capacity error 12.8 %.** The L3 estimate under-reads the nominal 20 MiB because the pointer chase shares the L3 with the rest of the machine and reaches its soft, contended knee before the full capacity; the match is nonetheless well inside the factor-of-two tolerance (§6.6). This is the framework’s first full L1/L2/L3/DRAM validation on x86, and it is reported with its **provenance**: it is a property of the **2 MiB huge-page run**, not of the default 4 KiB-page behaviour above.

Table 5: Discovered hierarchy vs. per-core ground truth (Intel i5-13450HX, performance core; 2 MiB huge pages, `--runs 3 --max-mb 512 --huge-pages`).

Level	Detected capacity	Median latency	p5–p95	Documented (per P-core)	Note
L1 Cache	55.7 KiB	1.59 ns	1.58–1.96 ns	48 KiB	+16.0 % — matches
L2 Cache	1.2 MiB	4.75 ns	4.74–5.11 ns	1.25 MiB	–1.5 % — matches
L3 Cache	13.9 MiB	22.94 ns	16.74–55.37 ns	20 MiB (shared)	–30.4 % (0.52 oct) — matches
DRAM	—	123.25 ns	103.49–129.31 ns	—	page-walk cost lifted by huge pages

The productive level counter is now stable and correct, but the estimators are still not unanimous. With huge pages the segmenter finds four bands and, decisively, the framework’s productive counter — **K-Means + Silhouette** — **is now stable at the correct four levels across all three sweeps (mean 4.0, std 0.00)**, where under 4 KiB pages it was unstable (modal 2, mean 2.67). DBSCAN also

lands on a stable four. The ensemble as a whole, however, does **not** reach the clean unanimity seen on the M1: the independent change-point cost-knee and the Elbow method still cut at **2** (the fast-vs-slow L1+L2 vs memory split), and GMM over-counts to a modal **5**. So huge pages fix the *productive* count and the L3 recall, but “one counter correct on every architecture” remains too strong — the closely-spaced deep bands are still a regime where the count-free cross-checks scatter (Literature Review §5).

Table 6: Model selection — Elbow and Silhouette disagree (Fig. 8). Unlike the M1 (Table 2, where both criteria select $k = 3$), on the huge-page Intel sweep the two automatic model-selection criteria still choose **different** cluster counts — even though the L3 is now resolved and the Silhouette count is stable:

Criterion	Selected k
K-Means inertia (Elbow)	2
Silhouette score	4 (score 0.935)

The K-Means inertia elbows at $k = 2$ — the fast-vs-slow (L1/L2 vs. memory) split every run agrees on — whereas the Silhouette score peaks sharply at $k = 4$ (score 0.935, up from 0.860 on 4 KiB pages: the recovered L3 plateau makes the four-way partition cleaner). This persistent disagreement, visualised in Fig. 8, is the model-selection form of the *not-unanimous* x86 estimator ensemble quantified in Table 7, and the direct contrast with the M1’s fully unanimous agreement (Table 2 / Fig. 6): huge pages make the productive Silhouette count both accurate and stable, but they do **not** make the Elbow criterion agree with it.

Table 7: Level-count estimators — comparison and stability (Intel i5-13450HX, 2 MiB huge pages, 3 sweeps; expected 4). Compare with the M1’s Table 3, where all five estimators agreed at three with zero variance; here the *productive* counter is now accurate and perfectly stable, but the ensemble is not unanimous.

Rank	Method	Mean levels	Std (stability)	Modal
1	K-Means + Silhouette	4.00	0.00	4
2	DBSCAN	4.00	0.00	4
3	GMM + Silhouette	4.33	0.47	4
4	Change-point (cost-knee)	2.00	0.00	2
5	K-Means + Elbow	2.00	0.00	2

With huge pages the productive **K-Means + Silhouette** counter is both accurate and perfectly stable at the expected four (mean 4.00, std 0.00), reversing the 4 KiB result where it was unstable at a modal two; DBSCAN now agrees, and even GMM’s modal count reaches four. The dissent is confined to the two *fast-vs-slow* counters — the change-point cost-knee and the Elbow — which still cut at two. So the honest reading shifts from the 4 KiB “deeper structure not reliably counted” to “**the productive counter now recovers all four levels, stably, but two independent cross-checks still under-count**” — a not-unanimous ensemble, distinct from the M1’s five-way agreement.

Table 8: Change-point level count vs. manual PELT penalty (Intel, 2 MiB huge-page sweep). Unlike the M1 (Table 4, where the count slid from six to three as the penalty rose), the Intel huge-page min-curve segments into **four** bands across essentially the whole penalty range: the L1 / L2 / L3 /

DRAM shape is robustly present in the curve, so it is the *cross-estimator* agreement (Table 7), not the penalty, that remains partial.

Penalty	1	2	3	4	6	8	10
Levels	5	5	4	4	4	4	4

Ground-truth validation on Windows — now reading true per-core sizes. The framework’s automatic accuracy initially read **0 %**, but this was an artifact of the Windows ground-truth path, not of the measurement. The legacy Win32_CacheMemory WMI class reports *per-socket aggregate* cache sizes (it returns L1 = 288 KiB = 6 × 48 KiB and L2 = 7680 KiB = 6 × 1.25 MiB), whereas the single-core probe measures *per-core* caches; a per-core measurement cannot match a summed-over-cores figure. `validation.py` was therefore changed to query true per-core sizes via `GetLogicalProcessorInformationEx` (RelationCache) — keeping the data/unified caches whose processor-affinity mask serves CPU 0 and reporting L1 = 48 KiB, L2 = 1.25 MiB, L3 = 20 MiB on this SKU, with the WMI aggregate retained only as a labelled fallback. Against this corrected per-core ground truth all three detected caches match within the factor-of-two tolerance under the 2 MiB huge-page run: L1 (55.7 vs 48 KiB, +16.0 %), L2 (1.2 vs 1.25 MiB, -1.5 %), and — now that the huge-page control has unmasked it — **L3 (13.9 vs 20 MiB, -30.4 %, 0.52 octaves)**. Recall is therefore **100 % (3/3 documented caches) at 100 % precision** (no false-positive knees; F1 = 1.00, mean absolute capacity error 12.8 % over three sweeps), up from the 4 KiB baseline’s 66.7 %/66.7 % where the L3 was masked and the ~3.5 MiB TLB knee was a false positive. (Both the per-core ground-truth fix and the huge-page control were needed: the corrected per-core query alone lifted recall from a spurious 0 % — the legacy Win32_CacheMemory path reports *per-socket aggregate* sizes the single-core probe cannot match — to 66.7 %, and huge pages then carried it to 100 %.) This also closes a real portability gap — macOS and Linux already read per-core caches via `sysctl/sysfs`, and Windows now does too.

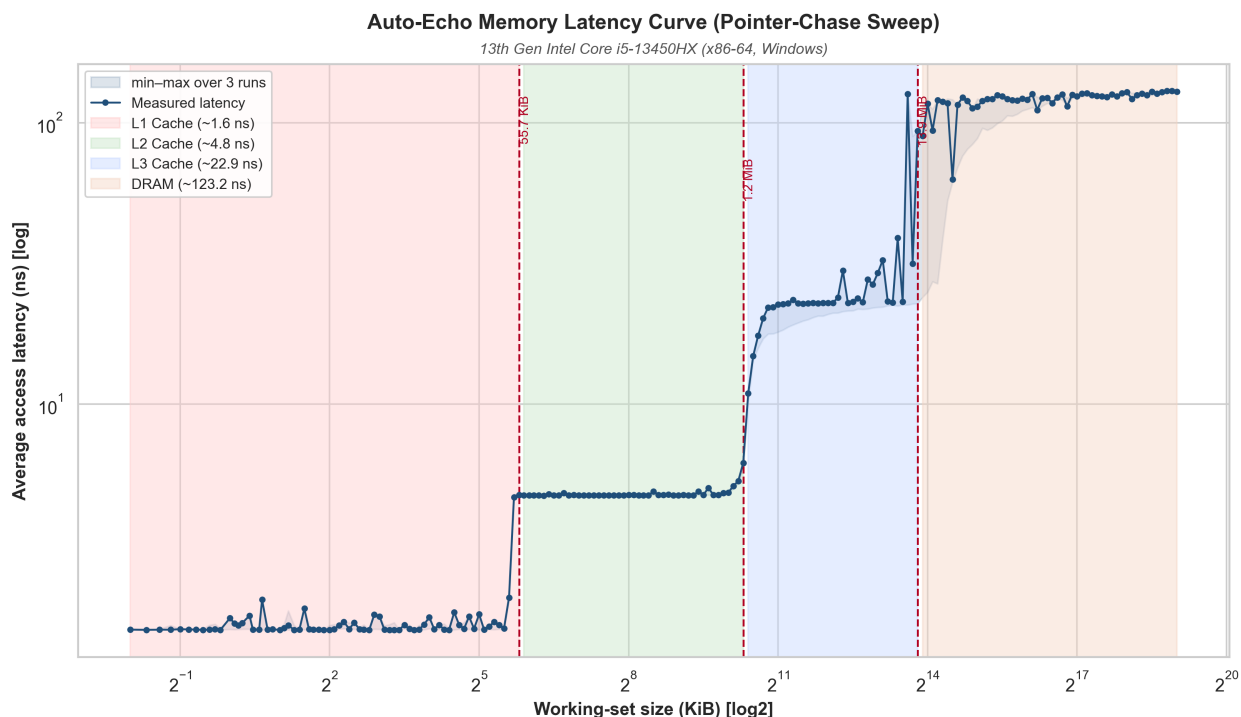


Fig. 7. Pointer-chase latency vs. working-set size on the Intel i5-13450HX performance core under a **2 MiB huge-page** allocation (minimum over three sweeps; the light band is the min-max spread). Four plateaus are now resolved — L1 (~48 KiB), L2 (~1.25 MiB), L3 (~14 MiB) and DRAM (~123 ns) — because the huge-page TLB reach removes the page-walk cost that, under the default 4 KiB pages, saturated the curve at ~143 ns and masked the L3 (§6.3).

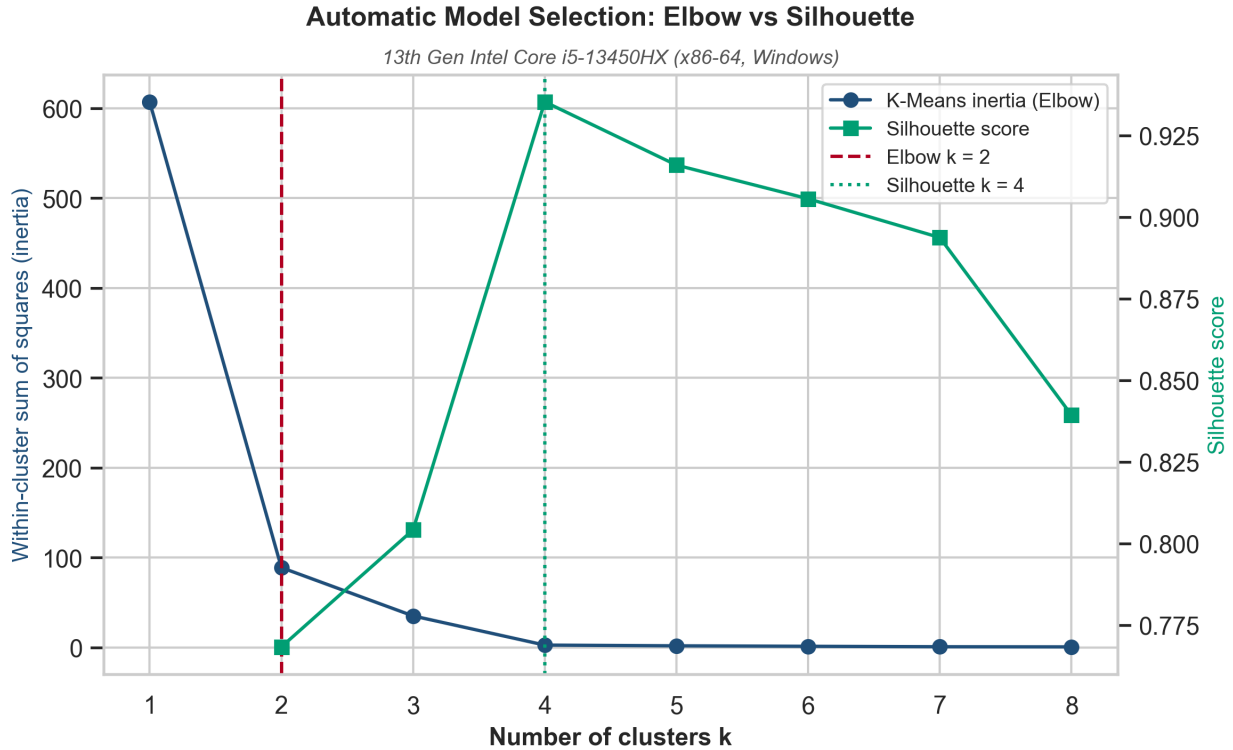


Fig. 8. Automatic model selection on the Intel i5-13450HX (2 MiB huge-page run): the K-Means inertia elbow selects $k = 2$ while the Silhouette score peaks at $k = 4$ — the two criteria still **disagree** even though the L3 is now resolved and the Silhouette count is stable, the model-selection signature of the not-unanimous x86 estimator ensemble. Contrast Fig. 6, where both criteria agree at $k = 3$ on the M1.

6.4 Apple M5 (ARM64) — to be measured

An Apple M5 part is a newer Apple-silicon generation; running Auto-Echo on it would add a second, independent ARM64 data point across CPU generations and a further test of architecture-agnosticism, complementing the M1 (§6.2).

Table 9: Discovered hierarchy vs. ground truth (Apple M5 — placeholder).

Level	Detected capacity	Median latency	p5–p95	Ground truth	Error
L1 Cache	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
L2 Cache	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
L3 Cache	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
DRAM	—	<i>TBD</i>	<i>TBD</i>	—	—

Validation accuracy: *TBD*; mean absolute capacity error: *TBD*.

6.5 Cross-platform summary and architecture-agnostic behaviour

The level count is never hard-coded: it is chosen from the data, so it adapts to whatever hierarchy the machine exposes. Across the **two real machines** now measured, the *same* code path recovers the resolvable cache boundaries on both a 128-byte-line ARM64 core and a 64-byte-line x86-64 core: **three** levels on the Apple M1 (L1, a merged L2/SLC band, DRAM; §6.2) and, under a 2 MiB huge-page control, the **full L1 (~48 KiB) / L2 (~1.25 MiB) / L3 (~14 MiB) / DRAM** hierarchy on the Intel i5-13450HX (§6.3). This is genuine cross-architecture evidence for the memory hierarchy — the core of the architecture-agnostic claim.

Two honest qualifications follow from the real x86 run. First, the earlier expectation (from *synthetic* Intel/AMD/VM curves) that the method would cleanly recover **four** L1/L2/L3/DRAM levels on x86 held **only under a huge-page control**, not with the default 4 KiB pages: on 4 KiB pages TLB/page-walk latency masks the 20 MiB L3 (§6.3), and the deep hierarchy separates only once the 2 MiB large-page allocation removes the page-walk cost — so the four-level x86 result is real but carries the huge-page provenance. Second, the level *count* is stable **and unanimous** on the M1, whereas on the Intel part huge pages make the *productive* K-Means + Silhouette counter stable and correct (four levels) but the estimator ensemble is still **not unanimous** — the Elbow and change-point cost-knee cross-checks under-count to two (§6.3) — so “one counter is correct, and every counter agrees, on every architecture” is known to be too strong. The synthetic curves (Fig. 9) should therefore be read as *method verification* — showing the counting machinery adapts to a given staircase shape — not as evidence about real x86 behaviour, which §6.3 supersedes.

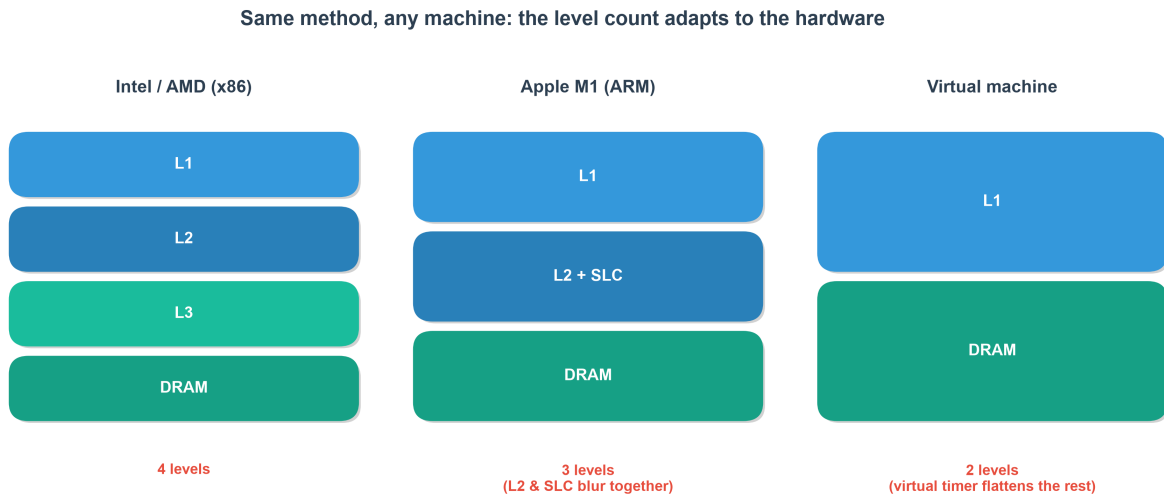


Fig. 9. Method verification on **synthetic** staircase curves: the automatic counter adapts its level count to the input shape — four levels on an idealised x86 profile, three on an M1-shaped profile, two on a flattened VM profile. These are modelled curves, not measurements; the real Intel result (§6.3) reaches the idealised four-level x86 shape only under a 2 MiB huge-page control — the default 4 KiB pages exhibit TLB effects the synthetic profile omits — and should be read in preference to it.

Metric	Apple M1 (ARM64)	Intel i5-13450HX (x86-64)	Apple M5 (ARM64)
Cache line size	128 B	64 B	<i>TBD (128 B)</i>
Levels resolved (automatic)	3 (L1 / L2+SLC / DRAM)	4 (L1 / L2 / L3 / DRAM), 2 MiB huge pages; L3 masked on 4 KiB	<i>TBD</i>
Caches matched (per-core ground truth)	2/2 documented	3/3 documented (huge pages); 2/3 on 4 KiB	<i>TBD</i>
Count stability across 3 sweeps	unanimous, std 0	productive counter stable at 4 (std 0), ensemble not unanimous (Elbow/cost-knee = 2)	<i>TBD</i>
Naive baseline (with <code>clflush</code>)	n/a (no ARM flush)	2 tiers — fails to resolve (L1-residency + timer grid)	<i>TBD</i>

With two real curves now in hand, a combined cross-machine overlay (`compare_curves.py`) plots the M1’s 128 KiB / 12 MiB knees against the Intel core’s 48 KiB / 1.25 MiB / ~14 MiB knees on one log-log axis (Fig. 10, refreshed from the 2 MiB huge-page Intel sweep). Both are flat in L1; the M1 then carries a single high **L2+SLC** shelf (~9 ns) where the Intel shows a distinct **L1→L2→L3** staircase up to the recovered ~14 MiB L3; both climb to a ~123–130 ns DRAM plateau. This is direct visual evidence for architecture-agnostic recovery of the hierarchy across ARM64 and x86-64. The Apple M5 curve would extend it to three machines.

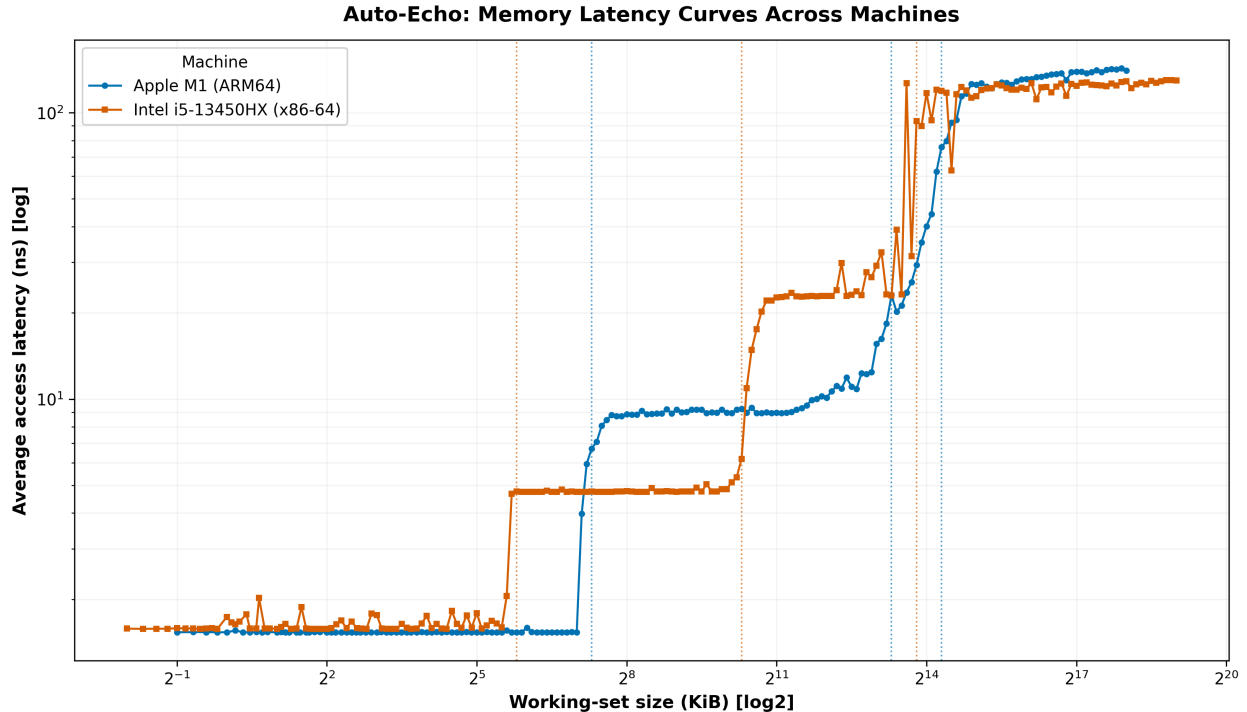


Fig. 10. Auto-Echo latency curves for the Apple M1 (ARM64) and Intel i5-13450HX (x86-64, 2 MiB huge-

page run) on one axis, each machine’s detected cache boundaries marked (dotted). L1, L2 and — on the Intel, once huge pages lift the 4 KiB page-walk masking — the L3 are recovered across both ISAs (§6.3).

Capacity-estimator comparison. The systematic +23% edge bias (§6.2) prompted evaluating two alternatives to the default *plateau-edge* estimator (capacity = the largest working-set size still served at plateau latency): the transition *midpoint* (geometric mean of the last-fit and first-miss sizes) and an *onset* estimator (the last size still on the plateau floor before latency departs by 10%). Absolute error against per-core ground truth:

Level (per-core GT)	Edge (default)	Midpoint	Onset	Hybrid
Apple M1 — L1 (128 KiB)	+23.0%	+27.4%	+0.0%	+0.0%
Apple M1 — L2+SLC (12 MiB)	+16.1%	+20.2%	−74.7%	+16.1%
Intel — L1 (48 KiB)	+16.0%	+20.1%	−98.3%	+16.0%
Intel — L2 (1.25 MiB)	−1.5%	+2.0%	−75.4%	−1.5%

The result is a clear **accuracy-versus-robustness trade-off**. The onset estimator is *exact* on the M1 L1 (+0.0%, recovering the nominal 128 KiB) — which confirms that the +23% is a genuine soft-knee artefact and that the physical knee onset does sit at the documented capacity — but it is **catastrophically unstable elsewhere** (−57% to −98%). A floor-departure threshold is meaningful only on a *flat* plateau with a *sharp* knee, so it collapses on the M1’s continuously-sloping merged L2+SLC band and on the Intel plateaus, where early-plateau noise trips it almost immediately; no threshold, smoothing kernel, or sustained-departure rule rescues a sloping plateau. The midpoint is strictly worse than the edge on every level. The **plateau-edge estimator is therefore retained as the default**: its bias is consistent (+16–23%), one-signed (upward), and never catastrophic, which for an automatic, threshold-free tool is worth more than an estimator that is occasionally exact and usually wrong. This also fixes the honest reading of the headline accuracy — Auto-Echo reliably places a *detected boundary within one octave* of each documented cache, with a characterised upward bias, rather than recovering the nominal capacity to a tight percentage. The **flat-plateau-gated hybrid** — onset where the plateau is flat, edge otherwise — is now implemented as an opt-in estimator (`--capacity-method hybrid`, rightmost column): it recovers the M1 L1 **exactly** (128 KiB, +0.0%) and, because the M1’s sloping L2+SLC band and every Intel plateau fail the flatness gate, falls back to the stable edge there, so it is never catastrophic. It is opt-in rather than the default only because it shifts the reported M1 L1 capacity from 157 KiB to the nominal 128 KiB; the default remains the edge estimator, and the hybrid is available for a user who wants nominal capacities on well-separated hierarchies.

6.6 Threats to validity

Following the structure of the reference paper’s own threats section [1]:

- **External validity (generalisation).** Results now span *two* machines (an Apple M1 performance core and an Intel Raptor Lake performance core), which demonstrates architecture-agnostic recovery of the hierarchy across ARM64 and x86-64 — the inner L1/L2 on both, and, under a 2 MiB huge-page control, the full L1/L2/L3/DRAM on the Intel part (§6.3). Two limits remain: the deep x86 result requires huge pages (the default 4 KiB behaviour still masks the L3), and the level *count*, though stable and correct for the productive Silhouette counter on both machines, is cross-estimator *unanimous* only on the M1. An Apple M5 data point is still outstanding.
- **Construct validity (what is measured).** The pointer chase measures *load-to-use* latency of a serialised dependent chain, which includes base pipeline cost — so the ~1.53 ns L1 figure is not

directly comparable to the paper’s `rdtscp`-dominated 10–25 ns. Absolute values are best read as *relative* differences between tiers (as the paper itself argues in §6.2).

- **Confounding — TLB and page walks.** As the working set grows, the number of distinct pages touched grows with it; deep-plateau latency therefore includes DTLB-miss and page-walk cost, which page alignment does *not* remove. On the M1 this blurs the mid-cache (~10–14 MB) region; on the Intel part it is decisive with the default 4 KiB pages — the page-walk penalty saturates the curve at ~143 ns by ~4 MiB and **masks the 20 MiB L3 entirely** (§6.3). This confounder is now *directly addressed*: the implemented 2 MiB huge-page control extends the TLB reach, drops the deep plateau to ~123 ns, and **recovers the L3** (13.9 MiB, within a factor of two of 20 MiB), lifting recall to 3/3 (§6.3). It remains a genuine limit in that the full x86 hierarchy is separable only under huge pages, not the default 4 KiB pages, and performance-counter corroboration is still desirable.
- **SLC attribution.** Automatic model selection reports the L2 and SLC as one merged mid-band; the finer split (forced only at an explicit penalty ~ 4) is *consistent with* a distinct L2 and the M1 SLC but is not uniquely attributable to it without per-core-type, cross-core and counter-based experiments.
- **Measurement bias.** Latency is the *minimum* over repeats (a lower envelope that hides variability); and although the sweep order is now seed-randomised to decorrelate size from thermal drift, sustained thermal throttling remains a possible bias on long sweeps.
- **Validation metric — precision as well as recall.** Detected knees are assigned to documented caches by *optimal* one-to-one (Hungarian) matching rather than greedy nearest-first (which can undercount when adjacent caches fall within tolerance), and the framework reports **precision** (the fraction of detected knees that are real caches) alongside **recall** (the fraction of documented caches found). This directly penalises false-positive levels: on the Intel part’s **default 4 KiB** run the ~3.5 MiB knee is a TLB-transition artefact, so precision is **2/3 (67%)** even though the L1 and L2 knees are correct — a distinction a recall-only “accuracy” conceals, and precisely the failure mode a *discovery* tool must expose. Under the 2 MiB huge-page run that false positive disappears (the third knee is now the real L3), so precision and recall both reach 3/3 (100%; §6.3). A factor-of-two match remains permissive (a 200 KiB detection would still match a 128 KiB L1) and ground truth is OS-reported; multiple tolerances ($\pm 10/25/50\%$) and an external `lmbench` cross-check are the remaining planned checks. An **onset**-based capacity estimator (§6.2) — first departure from the plateau median rather than the plateau edge — could reduce the systematic *upward* bias in the reported capacities, at some cost in robustness.
- **Ground truth on Windows (fixed).** The legacy Windows `Win32_CacheMemory` path reported *per-socket aggregate* cache sizes, not the per-core sizes the single-core probe measures, so the automatic accuracy metric read a spurious 0 % on the Intel part despite correct L1/L2 knees. `validation.py` now reads true per-core sizes via `GetLogicalProcessorInformationEx` (with the WMI aggregate kept only as a labelled fallback), so the accuracy metric on the Intel part is correct — 66.7 % on the default 4 KiB pages and 100 % under the 2 MiB huge-page run (§6.3). The macOS/Linux paths use per-core `sysctl/sysfs` values and were already unaffected.
- **Conclusion validity.** On the M1 the level count is stable across sweeps and all five estimators agree on three levels (Table 3). On the Intel part the 2 MiB huge-page run makes the *productive* K-Means + Silhouette counter stable and correct at four levels (std 0; Table 7), but the estimator ensemble does **not** reach M1-style unanimity — the Elbow and change-point cost-knee cross-checks under-count to two (§6.3). Cross-estimator agreement is therefore **machine-dependent, not universal**; the robust cross-machine claim is that the productive counter recovers the boundaries (L1/L2 on both, plus the L3 on x86 under huge pages), while full five-way estimator agreement is demonstrated only on the M1.

7. Discussion & Future Work

Auto-Echo demonstrates accurate, architecture-agnostic hierarchy discovery on both Apple Silicon and x86-64 — fully unprivileged on the M1, and, on x86, unprivileged for the L1/L2 result with the deep L3 map requiring only the ordinary “Lock pages in memory” user right (no kernel module or driver). Remaining directions:

- **Huge-page control to unmask the L3 (implemented and executed).** The default-4 KiB Intel run (§6.3) shows TLB/page-walk latency saturating the curve before the 20 MiB L3 is reached. The gated 2 MiB large-page allocation path (`--huge-pages / AUTOECHO_HUGEPAGES=1; VirtualAlloc(MEM_LARGE_PAGES)` on Windows, with a graceful fall-back to 4 KiB pages) has now been **run to completion** after granting the Windows `SeLockMemoryPrivilege` (“Lock pages in memory”) right: it cuts the page-walk cost sharply, exposes the L3 plateau, and converts the x86 result from “L1/L2 recovered” to a **full, validated L1/L2/L3/DRAM map** (3/3 caches, 100 % recall/precision; §6.3). What remains is to reproduce it on a machine where the L3 is not shared under contention (to test whether the ~30 % L3 under-read narrows) and to add performance-counter corroboration. (The runtime-calibration path on the invariant TSC is confirmed on real x86 — 0.383 ns/tick on the i5-13450HX.)
- **A third machine (Apple M5).** An Apple M5 part would add a second, independent ARM64 data point across Apple-silicon generations. (The per-core Windows ground-truth query via `GetLogicalProcessorInformationEx`, previously listed here as future work, is now implemented, so the automatic accuracy metric is valid on Windows — §6.3, §6.6.)
- **Cross-machine comparison figure.** A helper (`compare_curves.py`) overlays the per-machine latency curves (`wss_curve.csv`) on a single log-log axis with each machine’s detected cache boundaries annotated. Comparing the M1’s 128 KiB / 12 MiB knees against an x86 machine’s distinct L1/L2/L3 knees on one plot provides direct visual evidence for the architecture-agnostic claim.
- **External cross-check against lmbench.** A converter (`crosscheck_lmbench.py`) turns `lat_mem_rd` output [9] into the same curve format, so Auto-Echo’s probe can be overlaid against the established tool on identical hardware — validating the measurement against trusted prior art rather than only self-consistency.
- **Second dimension (stride sweep).** Sweeping stride as well as size would recover **cache line size and associativity**, extending Auto-Echo from a 1-D slice to the full memory mountain [11].
- **Automatic model selection (delivered).** Earlier drafts left the PELT penalty as a fixed hyperparameter. It is now removed: the level count is set automatically by Silhouette model selection and the boundaries by penalty-free change-point localisation, so no manual tuning knob remains.
- **Statistical confidence.** Repeated sweeps would let boundaries be reported with confidence intervals rather than point estimates.

8. Conclusion

Beginning from a naive echolocation probe whose empirical failure on modern hardware was analysed in detail, this project derived and implemented a principled alternative: a portable, flush-free working-set-size pointer-chase probe with batch-amortised timing, feeding an automatic, penalty-free

level-discovery stage in which clustering counts the levels and change-point localises them, validated against live OS ground truth. On an Apple M1 all five estimators agree on three levels and the framework recovers the L1 and L2 capacities to within 0.3 octaves (both documented caches matched within a factor of two); the 12 MiB L2 and the OS-unreported ~8 MB System-Level Cache resolve as a single mid-band, with their finer split a candidate sub-structure that further experiments must confirm (§6.6). A second real machine — an Intel Core i5-13450HX on Windows/x86-64 — was then measured with the identical pipeline. With the default 4 KiB pages the same unsupervised code path recovered its per-core L1 (~48 KiB) and L2 (~1.25 MiB) but TLB/page-walk latency masked the 20 MiB L3 and destabilised the automatic level count; enabling the 2 MiB huge-page control then removed the page-walk confounder and recovered the **full L1/L2/L3/DRAM hierarchy**, matching all three documented caches (3/3, 100 % recall and precision, mean absolute error 12.8 %) with the productive K-Means + Silhouette counter stable at four levels. Auto-Echo is therefore best characterised as a **portable framework validated on two ISAs** — the L1/L2 boundaries on both ARM64 and x86-64, and the complete L1/L2/L3/DRAM hierarchy on x86 under a huge-page allocation. Its results are reported with their provenance and residual honesty: the deep x86 map depends on 2 MiB huge pages rather than the default 4 KiB pages, the L3 reads ~30 % below nominal (still within a factor of two), and full cross-estimator unanimity is demonstrated only on the M1 — the Elbow and change-point cross-checks still under-count on x86. Adding an Apple M5 data point is the clearly-scoped remaining step. That the full hierarchy resolves *only* under huge pages is itself a finding, delimiting exactly where a user-space, single-core pointer chase can and cannot map a memory hierarchy, and what control lifts the limit.

9. References

- [1] V. Klimis, “Shouting at memory: Where did my write go?” in *Proc. 39th European Conf. on Object-Oriented Programming (ECOOP 2025)*, LIPIcs vol. 333, Art. 41, pp. 41:1–41:25, 2025. doi: 10.4230/LIPIcs.ECOOP.2025.41.
- [2] Apple Inc., “mach_absolute_time — Apple developer documentation,” 2024. [Online]. Available: https://developer.apple.com/documentation/kernel/1462446-mach_absolute_time
- [3] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD*, 2000, pp. 93–104.
- [4] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.
- [5] D. E. Knuth, *The Art of Computer Programming, Vol. 2*, 3rd ed. Addison-Wesley, 1997 (Fisher-Yates/Sattolo shuffle).
- [6] C. Truong, L. Oudre, and N. Vayatis, “Selective review of offline change point detection methods,” *Signal Processing*, vol. 167, 107299, 2020.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [8] R. H. Saavedra and A. J. Smith, “Measuring cache and TLB performance and their effect on benchmark run times,” *IEEE Trans. Computers*, vol. 44, no. 10, pp. 1223–1235, 1995.
- [9] L. McVoy and C. Staelin, “lmbench: Portable tools for performance analysis,” in *Proc. USENIX Annual Technical Conf.*, 1996 (the lat_mem_rd pointer-chase benchmark).
- [10] K. Yotov, K. Pingali, and P. Stodghill, “Automatic measurement of memory hierarchy parameters,” in *Proc. ACM SIGMETRICS*, 2005, pp. 181–192.
- [11] R. E. Bryant and D. R. O’Hallaron, *Computer Systems: A Programmer’s Perspective*, 3rd ed. Pearson, 2015 (the “memory mountain”).

- [12] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters (DBSCAN)," in *Proc. KDD*, 1996, pp. 226–231.
- [13] D. Johnson, "Apple M1 microarchitecture research," 2021. [Online]. Available: <https://dougallj.github.io/applecpu> (reverse-engineered Firestorm cache/SLC parameters, corroborating the ~8 MB System-Level Cache).